



Republic of the Philippines  
**Laguna State Polytechnic University**  
Province of Laguna

**College of Computer Studies**

**ITEC 204 – Data Structures and Algorithms**

|                                       |                                       |
|---------------------------------------|---------------------------------------|
| <b>Name: ROLDAN, JAZMIN CAMILLE V</b> | <b>Date: 09/24/2026</b>               |
| <b>Section: BS INFO TECH – 2A</b>     | <b>Instructor: MS. MAXYNE IGNACIO</b> |

- I. **Objective:** Write a short explanation of what your program is designed to do. (Minimum of 3 sentences)

**Answer:**

The program is an IT Automation Incident Ticket Manager that keeps track of incident tickets raised when automation bots fail during their tasks. It uses a singly linked list, where each ticket is stored as a node containing an Incident ID, the bot involved, a short description, and a pointer to the next ticket. Through a menu-driven interface, the user can add new tickets, display all active tickets, search for a ticket by its Incident ID, remove tickets that are already resolved, and view the total number of active tickets. The program loads ten sample tickets at startup so that all operations can be tested right away.

---

---

- II. **Program Operations:** Briefly explain how your program performs each operation. (Insert Specific Code Line and Your Explanation)

**Add Incident:**

**Code Line:**

```
new_ticket = Ticket(incident_id, bot, description)
if self.head is None:
    self.head = new_ticket
else:
    current = self.head
    while current.next:
        current = current.next
    current.next = new_ticket
self.size += 1
```

**Explanation:** The program first calls search\_ticket() to check if the Incident ID already exists, so duplicates are rejected. It then creates a new Ticket node. If the list is empty, the new node becomes the head. Otherwise, it walks through the list until it reaches the last node (the one whose next is None) and links the new ticket there, so tickets are added at the tail. Finally, the ticket counter size is increased by 1.



Republic of the Philippines  
**Laguna State Polytechnic University**  
Province of Laguna

**College of Computer Studies**

**Display Incidents:**

**Code Line:**

```
current, n = self.head, 1
while current:
    print(f"{n:<5}{current.incident_id:<15}{current.bot:<20}{current.description}")
    current = current.next
    n += 1
```

**Explanation:** If the head is None, the program prints that there are no active tickets. Otherwise, it starts at the head and traverses the list one node at a time, printing each ticket's number, Incident ID, bot, and description in a formatted table. It moves forward using `current = current.next` until it reaches the end of the list.

**Search Incident:**

**Code Line:**

```
current = self.head
while current:
    if current.incident_id.upper() == incident_id.upper():
        return current
    current = current.next
return None
```

**Explanation:** The program performs a linear search starting from the head. It compares each node's Incident ID with the one entered by the user, using `.upper()` so the search is not case-sensitive. If a match is found, the node is returned and its details are displayed; if the end of the list is reached without a match, it returns `None` and the program reports that the ticket was not found.

**Remove Resolved Incident:**

**Code Line:**

```
current, prev = self.head, None
while current:
    if current.incident_id.upper() == incident_id.upper():
        if prev is None:
            self.head = current.next
        else:
            prev.next = current.next
        self.size -= 1
        return True
    prev, current = current, current.next
return False
```

**Explanation:** The program traverses the list while keeping track of two pointers: `current` (the node being checked) and `prev` (the node before it). When the matching ticket is found, it is unlinked from the list. If it is the first node, the head is moved to the next node; otherwise, `prev.next` is pointed past the removed node. The counter `size` is decreased by 1. If no match is found, it returns `False`.



**Count Active Incidents:**

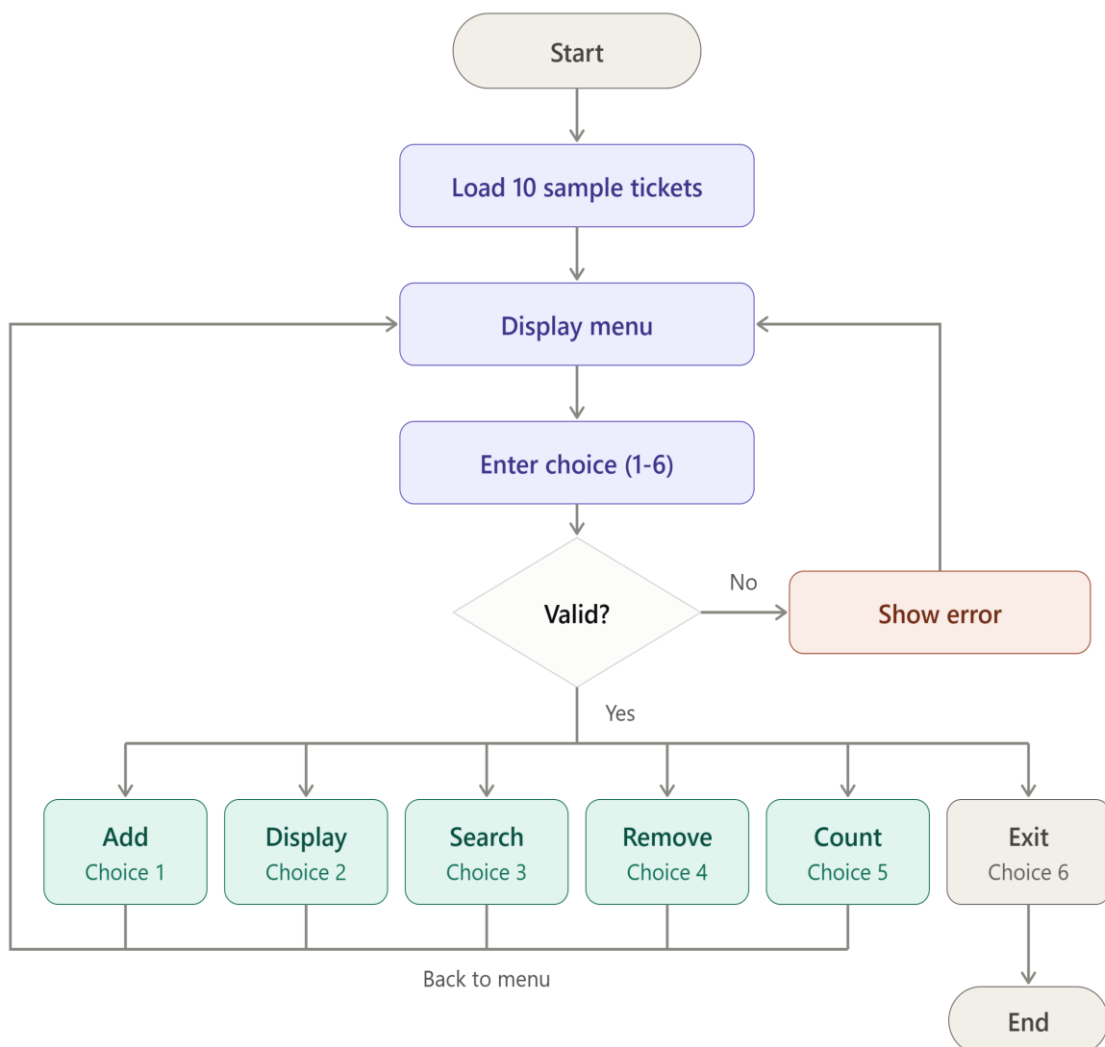
**Code Line:**

```
def count_tickets(self):  
    return self.size
```

**Explanation:** Instead of traversing the whole list every time, the program keeps a size variable that is increased when a ticket is added and decreased when a ticket is removed. Counting simply returns this value, which makes it fast ( $O(1)$ ).

---

III. **Flowchart:** Create a flowchart showing the overall process and logic of your program.





Republic of the Philippines  
**Laguna State Polytechnic University**  
Province of Laguna

**College of Computer Studies**

- IV. **Conclusion:** Briefly explain what you learned from the activity and how the linear data structure was applied.

Through this activity, I learned how a singly linked list works as a linear data structure and how it can be applied to a real-world scenario like managing incident tickets. Each ticket was stored as a node that points to the next one, so adding a ticket meant linking a new node at the tail, while displaying and searching required traversing the list from the head one node at a time. I also learned how removing a node works by re-linking the previous node to the node after the one being deleted, and how the head must be updated when the first node is removed. Keeping a size counter showed me that tracking information during insert and delete can make some operations faster. Overall, the activity helped me understand that linked lists are flexible because they can grow and shrink easily without shifting elements, although searching still takes linear time since there is no direct access by index.

---